

W10D04 — Environment Variables & Secrets Management

1. Analogi: Kunci Rumah vs Kunci Brankas

Bayangkan kamu punya dua tempat tinggal: rumah development dan apartemen production. Di rumah (dev), kamu taruh kunci cadangan di bawah keset — password WiFi-nya `dev123`, santai, tidak ada yang penting di sana. Di apartemen production, ada brankas berisi data nasabah dan API key yang kalau bocor bisa habis tagihan \$10.000 dalam semalam. Kunci brankasnya tidak kamu tulis di whiteboard, tidak kamu kirim via WhatsApp, tidak kamu commit ke Git.

Environment variables adalah mekanisme yang memisahkan *konfigurasi* dari *kode* — supaya kode yang sama bisa berjalan di dev maupun production, dengan credential yang berbeda, dan credential production tidak pernah menyentuh codebase.

2. Tiga Layer Masalah yang Diselesaikan

Layer	Masalah	Solusi
Separation of concerns	Kode tidak seharusnya tahu environment-nya sendiri	Kode hanya bertanya via <code>os.getenv()</code> — environment menjawab
Security	Credential tidak boleh masuk Git history atau Docker image	<code>gitignore</code> + <code>--env-file</code> + platform secrets
Portability	Satu codebase harus bisa jalan di dev dan prod	Satu base compose + override files + <code>.env</code> files berbeda

3. Struktur File

Pola yang digunakan: satu base compose + dua override + dua env file. Hanya `.env.example` yang boleh masuk Git — berisi template kosong tanpa nilai sensitif.

```
week-10/
■■■■ .env.example           ← template, BOLEH commit
■■■■ .env.dev               ← nilai dev, JANGAN commit
■■■■ .env.prod              ← nilai prod, JANGAN commit
■■■■ docker-compose.yml     ← base config
■■■■ docker-compose.dev.yml ← override dev
■■■■ docker-compose.prod.yml ← override prod
■■■■ app/
    ■■■■ config.py           ← centralized config loader
```

■ **Rule paling penting:** `.env.dev` dan `.env.prod` harus masuk `.gitignore`. Verifikasi dengan: `git check-ignore -v .env.dev` — kalau tidak ada output, file itu TIDAK di-ignore dan berbahaya.

4. Dev vs Prod Configuration

Setting	Dev (.env.dev)	Prod (.env.prod)
DEBUG	true	false

LOG_LEVEL	debug	warning
DB_URL	postgres://dev/riskdb	postgres://rds.aws.../prod
API_KEY	dev-key-dummy	★★★★★ (secret)
Redis port exposed	✓ 6379 (local debug)	✗ tidak di-expose
Bind mount code	✓ hot reload	✗ image-only, immutable
Restart policy	no	unless-stopped

5. Cara Compose Override Bekerja

Docker Compose **merge** dua file secara rekursif — bukan replace. Key yang sama di override file menimpa base. Key yang tidak ada di override dipertahankan dari base.

Command dev:

```
docker-compose -f docker-compose.yml -f docker-compose.dev.yml --env-file .env.dev up
```

Command prod:

```
docker-compose -f docker-compose.yml -f docker-compose.prod.yml --env-file .env.prod up -d
```

Kenapa tidak cukup docker-compose up saja? — Perintah tanpa flag hanya membaca `docker-compose.yml` dan `.env` default. Override file dan env file yang kita buat tidak akan terbaca.

6. docker-compose.yml (Base Config)

Base file berisi konfigurasi yang tidak berubah antara dev dan prod: service definitions, networks, volumes. Nilai variabel dibaca dari env file via sintaks `${VAR}`.

```
services:
  api:
    build: .
    environment:
      - APP_NAME=${APP_NAME}
      - DEBUG=${DEBUG}
      - API_KEY=${API_KEY}
      - REDIS_HOST=${REDIS_HOST}
      - REDIS_PASSWORD=${REDIS_PASSWORD}
    depends_on:
      - redis
    networks:
      - backend
  redis:
    image: redis:7-alpine
    command: >
      sh -c 'if [ -n "$$REDIS_PASSWORD" ];
        then redis-server --requirepass $$REDIS_PASSWORD --appendonly yes;
        else redis-server --appendonly yes;
        fi'
```

```
networks:
  - backend
volumes:
  - redis-data:/data
```

Kenapa \$\$REDIS_PASSWORD (double dollar)? — Di dalam command: YAML, Compose menginterpretasi \$VAR sebelum dikirim ke shell. \$\$ adalah escape: Compose ubah \$\$ → \$, lalu shell menerimanya sebagai variabel shell biasa.

7. Override Files

docker-compose.dev.yml:

```
services:
  api:
    volumes:
      - ./app:/app/app          # hot reload — kode dari host
      - ./app/model:/app/model
    ports:
      - "8000:8000"
    restart: "no"              # sadar kalau crash
  redis:
    ports:
      - "6379:6379"            # debug via RedisInsight
    restart: "no"
```

docker-compose.prod.yml:

```
services:
  api:
    # TIDAK ada bind mount — kode dari image, immutable
    ports:
      - "8000:8000"
    restart: unless-stopped
  redis:
    # TIDAK expose port — tidak perlu, attack surface
    restart: unless-stopped
```

8. Centralized Config Loader (config.py)

Single source of truth untuk semua environment variables. Tidak ada `os.getenv()` berserakan di seluruh codebase — semua dibaca dan divalidasi di sini saja. Menggunakan Pydantic **BaseSettings** yang otomatis membaca dari env vars dan .env file.

```
from pydantic import Field
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    app_name: str = "risk-scoring-api"
    app_version: str = "1.0.0"
    debug: bool = False
```

```

log_level: str = "info"
# Field(...) = WAJIB ada. Tidak ada default.
# Pydantic crash saat startup jika API_KEY tidak di-set.
api_key: str = Field(..., description="API Key, wajib diisi")
redis_host: str = "redis"
redis_port: int = 6379
redis_password: str = ""
model_path: str = "/app/model/model.joblib"
model_config = SettingsConfigDict(
    env_file=".env",
    env_file_encoding="utf-8",
    extra="ignore"
)
# Singleton – crash di sini jika config tidak valid (Fail Fast)
settings = Settings()

```

Kenapa Field(...) bukan Field(default="")? — String kosong adalah nilai *valid* di mata Pydantic. Dengan `default=""`, aplikasi tetap jalan tanpa API_KEY — noise berbahaya untuk field kritis. `Field(...)` berarti tidak ada default sama sekali: wajib diisi atau crash saat startup.

9. Script Verifikasi (verify_config.py)

Script untuk memverifikasi bahwa semua env vars ter-load dengan benar, tanpa mengekspos nilai sensitif ke log. Pattern ini penting sebelum deploy.

```

from app.config import settings
import sys

def verify_configuration():
    try:
        config_dict = settings.model_dump()
    except Exception as e:
        print(f"Gagal memuat konfigurasi: {e}")
        sys.exit(1)

    all_valid = True
    for key, value in config_dict.items():
        if key == "api_key":
            # Jangan pernah print nilai secret ke log
            print(f"{key}: {'SET' if value else 'MISSING'}")
            if not value:
                all_valid = False
        else:
            print(f"{key}: {value}")

    if not all_valid:
        sys.exit(1)

if __name__ == "__main__":
    verify_configuration()

```

10. Secrets di Production: Jangan Pakai File

Di production nyata, kamu **tidak menyimpan .env.prod di server**. File bisa bocor via backup, log, atau akses tidak sah. Platform modern menyediakan secrets management yang inject variabel langsung ke container:

Platform	Cara Inject Secrets
Railway	Dashboard → Variables
Render	Dashboard → Environment
Docker Swarm	<code>docker secret create</code>
Kubernetes	<code>kubectl create secret</code>
GitHub Actions	Repository → Settings → Secrets

11. Poin Penting yang Harus Diingat

- **.env.dev dan .env.prod tidak pernah masuk Git** — verifikasi dengan `git check-ignore -v .env.dev`.
- **.env.example boleh di-commit** — ini template tanpa nilai sensitif, membantu onboarding.
- **Compose override bersifat merge, bukan replace** — key yang sama ditimpa, key lain dipertahankan dari base.
- **--env-file harus eksplisit** — tanpa flag ini, Compose hanya baca .env default.
- **\$\$VAR di command: YAML** — double dollar untuk escape dari interpolasi Compose agar variabel dievaluasi oleh shell, bukan Compose.
- **Field(...) untuk variabel wajib** — Pydantic crash saat startup jika tidak ada, bukan saat request pertama. Fail fast.
- **Centralized config.py** — satu file sebagai single source of truth. Tidak ada `os.getenv()` berserakan.
- **Di production, gunakan platform secrets** — bukan file .env di server.
- **Jangan print secret ke log** — untuk `API_KEY` dan password, hanya print apakah SET atau MISSING.
- **Model path harus absolut di dalam container** — `/app/model/...` bukan `app/model/...`